Software Architecture and Design Specification

Electronics Store Inventory Management System

Project	Electronics Store Inventory Management System
Version	1.0
Team	H9
Authors	Shriya S R, Simret M Kotian, Ahana S S, Swara Potdar
Technology Stack	Python (Django), Django ORM, SQLite / PostgreSQL
Date	14-10-2026
Status	Draft / For Review

Team Members

No.	Name	SRN
1	Shriya S R	PES1UG24CS449
2	Simret M Kotian	PES1UG24CS456
3	Ahana S S	PES1UG24CS910
4	Swara Potdar	PES1UG24CS487

Revision History

Version	Date	Author	Change Summary
1.0	14-10-2026	Team H9	Complete architecture and design specification based on SRS v1.0 and the Phase 1 architecture extract

Approvals

Role	Name	Signature / Date
Course Coordinator	Prof. Ambika K	
Project Guide		

Table of Contents

1. Introduction

2. Document Overview

3. Architecture

4. Design

5. Appendices

Appendix A - Layered Application Architecture
Appendix B - Implemented URL Interface Design
Appendix C - Role and Database Scope

1. Introduction

1.1 Purpose

This document describes the architecture and design of the Electronics Store Inventory Management System. It explains how the system is organised into components, how those components work together, and how the requirements in the SRS are met.

1.2 Scope

The document covers the web application used by store users: login and logout, role-based access, adding, viewing, searching, updating and deleting electronic components, availability and low-stock identification, the dashboard, inventory reports, input validation, error handling, change tracking and persistent storage.

Out of scope (as in the SRS): online purchasing, payment processing, supplier payments, full accounting and demand forecasting.

1.3 Audience

Project team developers, testers (QA / Evaluator), the course coordinator, the project guide and evaluators.

1.4 Definitions

Term	Meaning
MVT	Model–View–Template, the pattern used by Django
ORM	Object Relational Mapper – Django layer that maps Python classes to database tables
RBAC	Role-Based Access Control – permissions are given to roles, roles are given to users
CSRF	Cross-Site Request Forgery – an attack using a forged request from another website
STRIDE	Threat model: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
ADR	Architecture Decision Record
ESIM-F / NF / SR	IDs of Functional, Non-Functional and Security requirements in the SRS

2. Document Overview

2.1 How to use this document

Section 3 gives the high-level architecture: goals, stakeholders, the component diagram, the chosen pattern, the technology stack, risks, traceability and security. Section 4 gives the detailed design: class diagram, sequence diagrams, API (URL) design, error handling and UX. Each component and design item refers back to the SRS requirement IDs.

2.2 Related Documents

Document	Description
SRS v1.0 (01_COMPLETE_SRS_IEEE_STYLE)	Functional, non-functional and security requirements, RTM and VFRs
Software Test Plan (STP) v1.0	Test strategy, schedule and traceability for this design
Test Cases (03_COMPLETE_TEST_CASES_15)	Planned test cases TC-xx referenced in this document
GitHub Project "SE Minproject"	Product backlog: 30 stories, story points and sprint plan

3. Architecture

3.1 Goals & Constraints

Goals

- **Correct inventory data** – stock quantities and component records are always accurate (ESIM-F-012, ESIM-NF-003).
- **Secure access** – only logged-in users can use the system, and only permitted roles can change data (ESIM-F-014, ESIM-SR-001..006).
- **Fast and simple to use** – 90% of list, search and CRUD requests within 2 seconds; usable without developer help (ESIM-NF-001, ESIM-NF-005).
- **Easy to maintain** – presentation, business logic, security and data access are separate modules (ESIM-NF-006).

Constraints

- Must be built with Python and Django; data stored in a relational database supported by Django.
- Must run in current Chrome, Edge and Firefox browsers (ESIM-NF-007).
- Academic project schedule: two one-week sprints (Sprint 1: 19–23 Oct, Sprint 2: 26–30 Oct 2026) and student hardware only.

3.2 Stakeholders & Concerns

Stakeholder	Main concerns
Inventory Administrator	Easy add / update / delete of components, managing users and roles, correct data
Store Staff	Quick search and clear availability information
Store Manager	Dashboard, low-stock alerts and reports for decisions
QA / Evaluator	Testability, traceability to requirements, security validation
Developers (Team H9)	Modularity, clear separation of layers, simple technology
Course Coordinator	Architecture follows the SRS and course templates

3.3 Component (UML) Diagram

The system follows a layered architecture. The browser talks to the presentation layer (Django templates, URLs and views). Every request passes through the security layer. Views call business-logic services, which use the Django ORM to read and write the database.

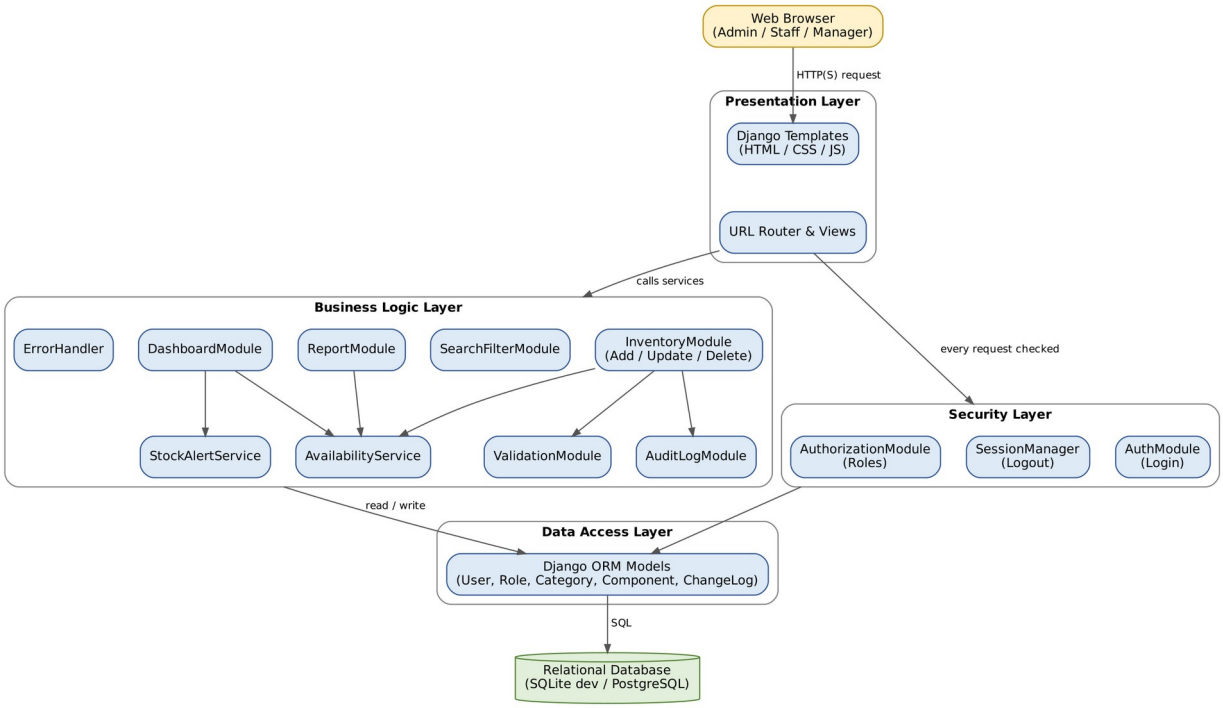


Figure 1 – Component diagram (layered architecture)

3.4 Component Descriptions

Component	Layer	Responsibility	Requirements
Django Templates (Web UI)	Presentation	Login page, dashboard, inventory list, component forms, search bar, reports page, messages	ESIM-NF-005, NF-007
URL Router & Views	Presentation	Receives HTTP requests, calls services, returns pages / redirects	All FRs
AuthModule	Security	Checks username and password (Django authentication) and creates a session	ESIM-F-001, SR-001, SR-003
SessionManager	Security	Ends the session on logout; protected pages need re-login	ESIM-F-002
AuthorizationModule	Security	Role / permission checks before add, update, delete and user management	ESIM-F-014, SR-002
InventoryModule	Business	Add, update and delete / deactivate components	ESIM-F-003, F-006, F-007
SearchFilterModule	Business	Search by ID / name and filter by category	ESIM-F-005
ValidationModule	Business	Server-side checks: required fields, unique ID, non-negative quantity, valid price	ESIM-F-012, SR-005
AvailabilityService	Business	Calculates availability from quantity and minimum stock level	ESIM-F-008
StockAlertService	Business	Finds components at or below minimum stock	ESIM-F-009
DashboardModule	Business	Totals: components, available stock, low-stock count	ESIM-F-010
ReportModule	Business	Generates, filters and exports inventory reports (CSV)	ESIM-F-011
AuditLogModule	Business	Records each change: item, action, user and time	ESIM-F-016
ErrorHandler	Business	Friendly error pages and messages; no technical details shown	ESIM-F-015

Component	Layer	Responsibility	Requirements
Django ORM Models	Data access	User, Role, Category, Component, ChangeLog; transactions	ESIM-F-013, NF-003
Relational Database	Data	SQLite (development) / PostgreSQL (deployment); backup and restore	ESIM-F-013, NF-008

3.5 Chosen Architecture Pattern and Rationale

Chosen: Layered architecture using Django's Model–View–Template (MVT) pattern, as a single web application (monolith).

- Clear separation of presentation, security, business logic and data access – directly satisfies ESIM-NF-006.
- Django already provides authentication, sessions, CSRF protection, password hashing, forms validation and an ORM, which reduces effort and security risk.
- A single application is easy to build, test and deploy for a four-person team in two one-week sprints.

Alternatives considered

Option	Decision	Reason
Microservices (separate services for auth, inventory, reports)	Rejected	Too complex for the project size and timeline; adds network and deployment overhead
Separate SPA front end (React) + REST API	Rejected for now	Needs two code bases; server-rendered Django templates are enough for the required pages
Layered Django MVT monolith	Accepted	Simple, secure defaults, matches the team's skills and the SRS constraints

ADR-01: Use Django MVT layered monolith

Status	Accepted
Context	Small team, two short sprints, need for security features and a relational database.
Decision	Build one Django project with separate apps/modules for accounts (auth & roles), inventory, reports and audit.
Consequences	Fast development and simple deployment. If the system grows, services can be split out later.

3.6 Technology Stack & Data Stores

Area	Technology
Language / Framework	Python 3, Django
Front end	Django templates, HTML5, CSS, a little JavaScript
Data access	Django ORM, Django migrations
Database	SQLite (development and testing); PostgreSQL / MySQL (larger deployment)
Security	Django authentication and sessions, PBKDF2 password hashing, CSRF middleware, Django groups/permissions for RBAC, HTTPS in deployment
Testing	Django TestCase / unittest, Selenium (UI), Locust or JMeter (performance), OWASP ZAP (security scan)
Tools	Git and GitHub (repository, Issues, Projects, Actions), VS Code

Main data stores (tables)

Table	Key fields

User	id, username, password (hashed), role
Role / Group	id, name, permissions
Category	id, name
Component	component_id (unique), name, category, description, unit_price, quantity, min_stock_level, is_active
ChangeLog	id, component, user, action (ADD / UPDATE / DELETE), timestamp, details

3.7 Risks & Mitigations

Risk	Mitigation
Wrong stock quantities lead to wrong availability and reports	Server-side validation, database transactions (transaction.atomic), unit tests at boundary values
Unauthorised users change inventory	Login required on all pages, role checks on every change, security tests SR-01 and SR-02
Data loss	Regular database backup script and tested restore (ESIM-NF-008)
Slow pages with large data	Database indexes on component_id, name and category; pagination of lists; performance test with 1000+ records
Tight sprint schedule	Core features in Sprint 1, testing and reports in Sprint 2; small stories with story points

3.8 Traceability to Requirements

Requirement	Component(s)	Test case
ESIM-F-001 Login	AuthModule	TC-AUTH-01
ESIM-F-002 Logout & session	SessionManager	TC-AUTH-02
ESIM-F-003 Add component	InventoryModule, ValidationModule	TC-INV-01
ESIM-F-004 View inventory	Views, Django ORM	TC-INV-02
ESIM-F-005 Search / filter	SearchFilterModule	TC-SEARCH-01
ESIM-F-006 Update component	InventoryModule	TC-INV-03
ESIM-F-007 Delete / deactivate	InventoryModule	TC-INV-04
ESIM-F-008 Availability	AvailabilityService	TC-AVAIL-01
ESIM-F-009 Low stock	StockAlertService	TC-LOW-01
ESIM-F-010 Dashboard	DashboardModule	TC-DASH-01
ESIM-F-011 Reports	ReportModule	TC-REP-01
ESIM-F-012 Input validation	ValidationModule	TC-VAL-01
ESIM-F-013 Persistent storage	Django ORM, Database	TC-DATA-01
ESIM-F-014 Authorization	AuthorizationModule	TC-SEC-01
ESIM-F-015 Error handling	ErrorHandler	TC-ERR-01
ESIM-F-016 Change tracking	AuditLogModule	TC-AUD-01
ESIM-NF-001 / 002 Performance	All layers, DB indexes, pagination	TC-PERF-01, TC-PERF-02
ESIM-NF-003 Reliability	Django ORM transactions	TC-REL-01

Requirement	Component(s)	Test case
ESIM-NF-006 Modularity	Layered structure (this document)	Code review
ESIM-SR-001..006 Security	Security layer, ValidationModule, logging config	TC-SEC-01..06

3.9 Security Architecture

Threat modelling uses STRIDE. Each threat is linked to a design control and the related security requirement.

Threat (STRIDE)	Example in this system	Control	Requirement
Spoofing	Someone uses another user's account	Django authentication, hashed passwords, session timeout, logout ends session	ESIM-SR-001, F-002
Tampering	Changing stock values with a forged request or bad input	CSRF tokens, server-side validation, ORM (no raw SQL)	ESIM-SR-004, SR-005
Repudiation	User denies making a stock change	ChangeLog records user, action and time	ESIM-F-016
Information disclosure	Passwords or secrets visible in DB or logs	Password hashing, no secrets in logs, HTTPS, DEBUG off in deployment	ESIM-SR-003, SR-006
Denial of service	Very many requests slow the system	Pagination, indexes, login throttling (planned)	ESIM-NF-001
Elevation of privilege	Staff user tries admin actions	RBAC checks in AuthorizationModule on every change	ESIM-F-014, SR-002

4. Design

4.1 Design Overview

The Django project is split into small apps, one per area, so each team member can work on separate code:

Django app	Contains
accounts	Login, logout, users, roles / permissions (AuthModule, SessionManager, AuthorizationModule)
inventory	Component and Category models, add / view / search / update / delete views, validation, availability and low-stock services
reports	Dashboard and report views, CSV export
audit	ChangeLog model and change-history view
core	Shared templates, error pages, messages and settings

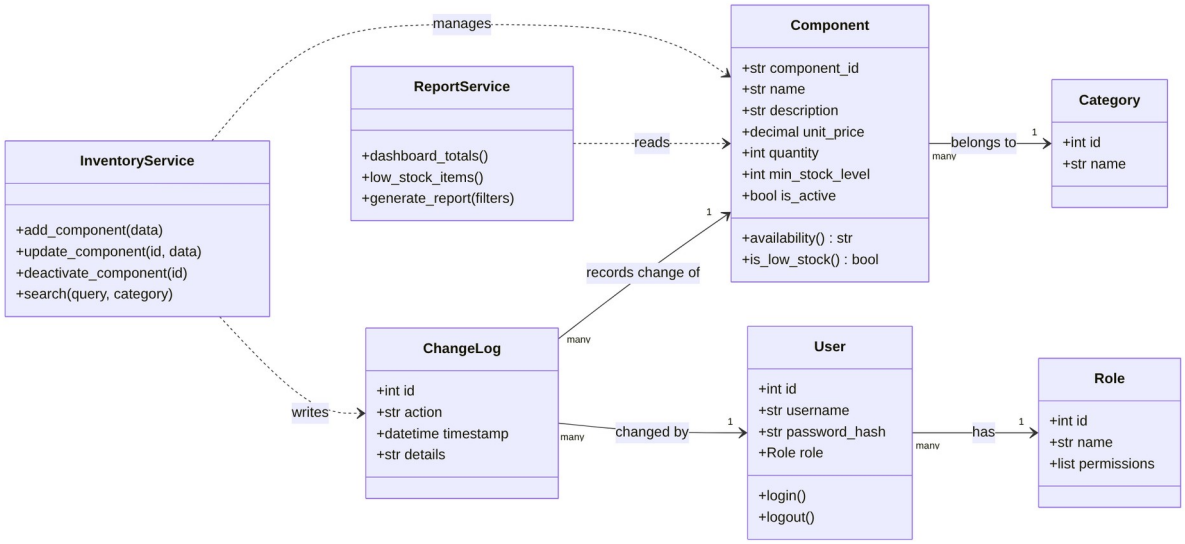


Figure 2 – UML class diagram (main models and services)

4.2 UML Sequence Diagrams

Flow 1: User login (ESIM-F-001)

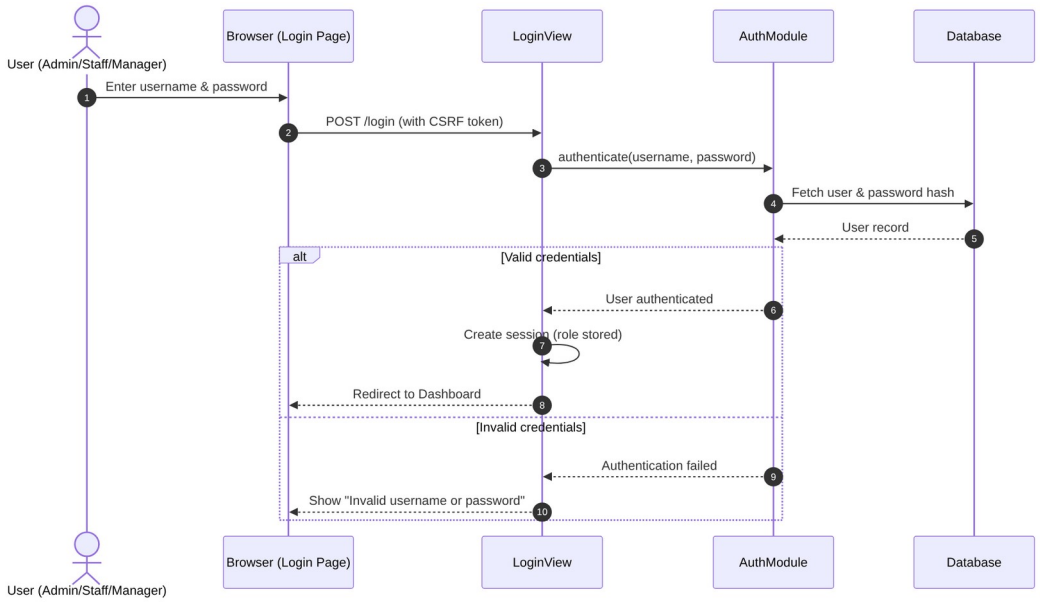


Figure 3 – Sequence diagram: user login

Flow 2: Add a new component (ESIM-F-003, F-012, F-014, F-016)

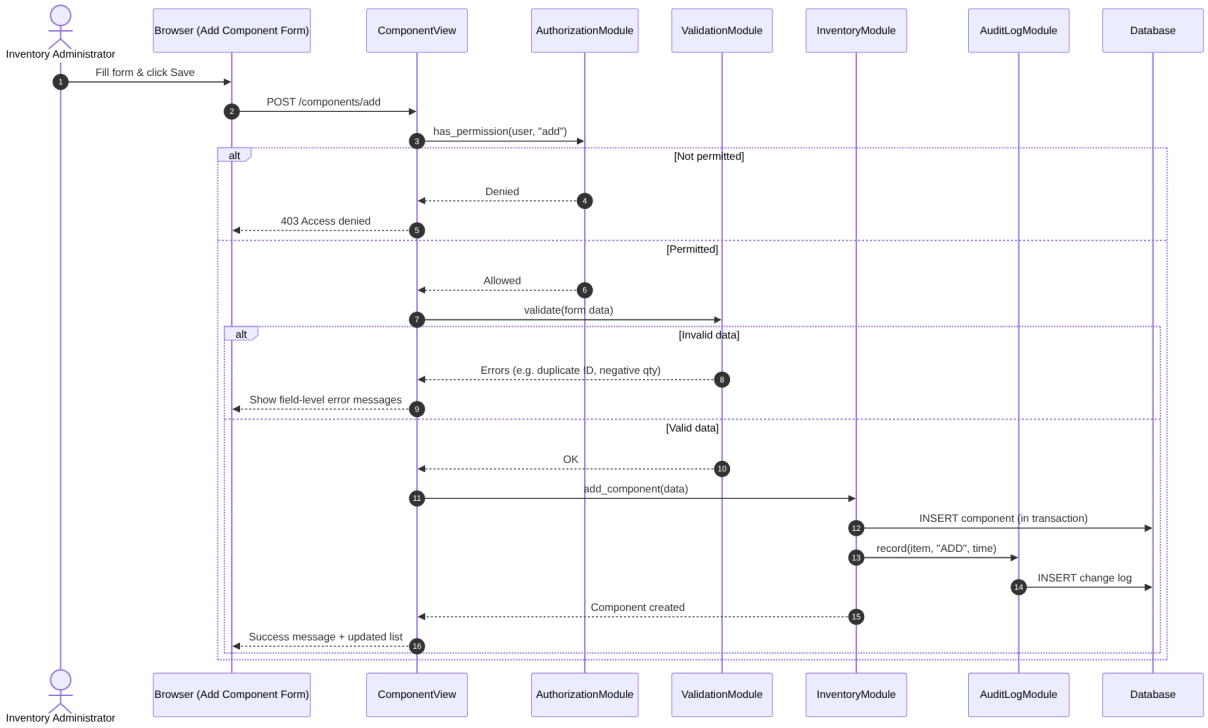


Figure 4 – Sequence diagram: add component with permission check, validation and change log

Flow 3: View dashboard and low-stock items (ESIM-F-009, F-010)

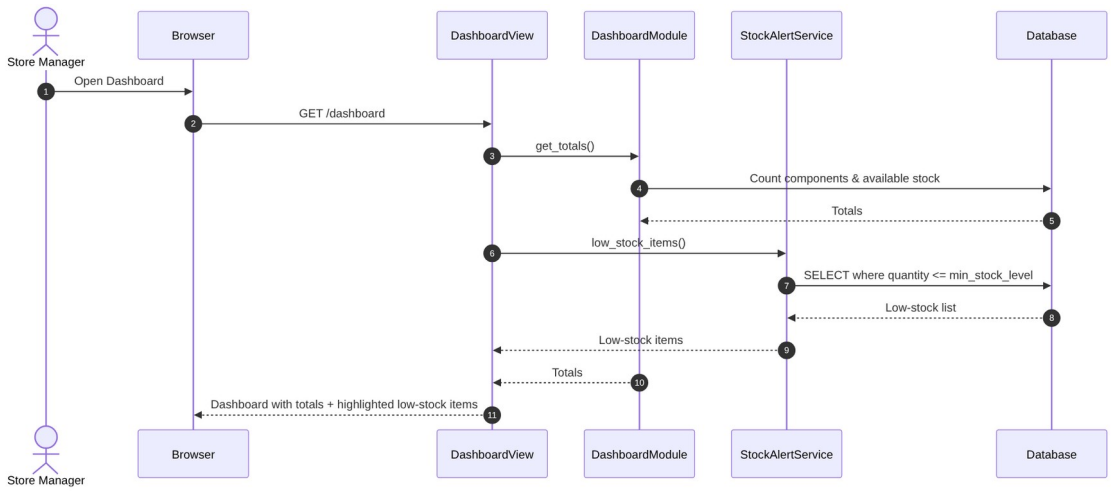


Figure 5 – Sequence diagram: dashboard and low-stock items

4.3 API Design

The application is server-rendered, so its interface is a set of URL endpoints handled by Django views. All POST requests need a valid CSRF token and (except login) a logged-in session.

Component 1: Accounts (AuthModule / SessionManager)

Endpoint	Method	Request	Response	Errors
/login/	POST	username, password, csrf token	Redirect to /dashboard/, session created	Invalid credentials message (form re-shown)
/logout/	POST	csrf token	Session ended, redirect to /login/	–

Component 2: Inventory (InventoryModule / SearchFilterModule)

Endpoint	Method	Request	Response	Errors
/components/	GET	q (ID / name), category (optional), page	List of components with availability	Empty list shows "No components found"
/components/add/	POST	component_id, name, category, description, unit_price, quantity, min_stock_level	Redirect to list + success message	400 validation errors; 403 not permitted
/components/<id>/edit/	POST	Changed fields	Updated component + success message	400 validation errors; 403; 404 not found
/components/<id>/delete/	POST	confirm = yes	Component deactivated / deleted	403 not permitted; 404 not found

Component 3: Reports (DashboardModule / ReportModule)

Endpoint	Method	Request	Response	Errors
/dashboard/	GET	–	Totals and highlighted low-stock items	302 to login if not logged in
/reports/	GET	category, availability (filters), format = html | csv	Report page or CSV download	403 if role not allowed

Endpoint	Method	Request	Response	Errors
/audit/	GET	component (optional)	Change history list	403 for non-admin

4.4 Error Handling, Logging & Monitoring

- **User messages:** validation errors are shown next to the field; success and failure messages use the Django messages framework.
- **Error pages:** custom 403, 404 and 500 pages; no stack traces when DEBUG is off (ESIM-F-015).
- **Data safety:** changes run inside database transactions, so a failure leaves records unchanged (ESIM-NF-003).
- **Logging:** Python logging records logins, failed logins and inventory changes; passwords and secret keys are never logged (ESIM-SR-006).
- **Monitoring:** response times measured during testing; uptime noted during the controlled test run (ESIM-NF-001, NF-004).

4.5 UX Design

Page	Main elements	Used by
Login	Username, password, Login button, error message	All users
Dashboard	Cards for total components, available stock and low-stock count; low-stock list highlighted	All users (mainly Manager)
Inventory list	Search bar, category filter, table with availability badge (In stock / Low stock), Edit / Delete buttons by role	All users
Add / Edit component	Simple form with clear labels and field-level errors	Inventory Administrator
Reports	Filters, report table, Download CSV button	Manager, Administrator
Change history	Table of item, action, user, time	Inventory Administrator

Design rules: consistent navigation bar, readable font sizes, colour plus text for status (not colour alone), confirmation before delete, and a layout that works in Chrome, Edge and Firefox.

4.6 Open Issues & Next Steps

- Appendix C records the current role policy; team approval is needed for policy changes.
- Current demo scope uses file-backed SQLite; no production database is configured or in scope.
- Future enhancements: email alerts for low stock, barcode scanning, PDF reports, supplier reorder suggestions.

5. Appendices

5.1 Glossary

Component, Inventory, Minimum Stock Level, Availability, MVT, ORM, RBAC, CSRF, STRIDE, ADR, RTM (see Section 1.4 and the SRS definitions).

5.2 References

- Team H9 SRS v1.0 – Electronics Store Inventory Management System
- IEEE 42010 – Architecture description; IEEE 830 – SRS guidelines
- Django documentation (authentication, security, ORM); OWASP Top 10

5.3 Tools

Graphviz and Mermaid (diagrams), draw.io, Git and GitHub, VS Code.

Appendix A — Layered Application Architecture

Supplementary architecture view, adapted from the supplied Phase 1 Architecture Diagram and corrected to match the implemented Django apps and model names.

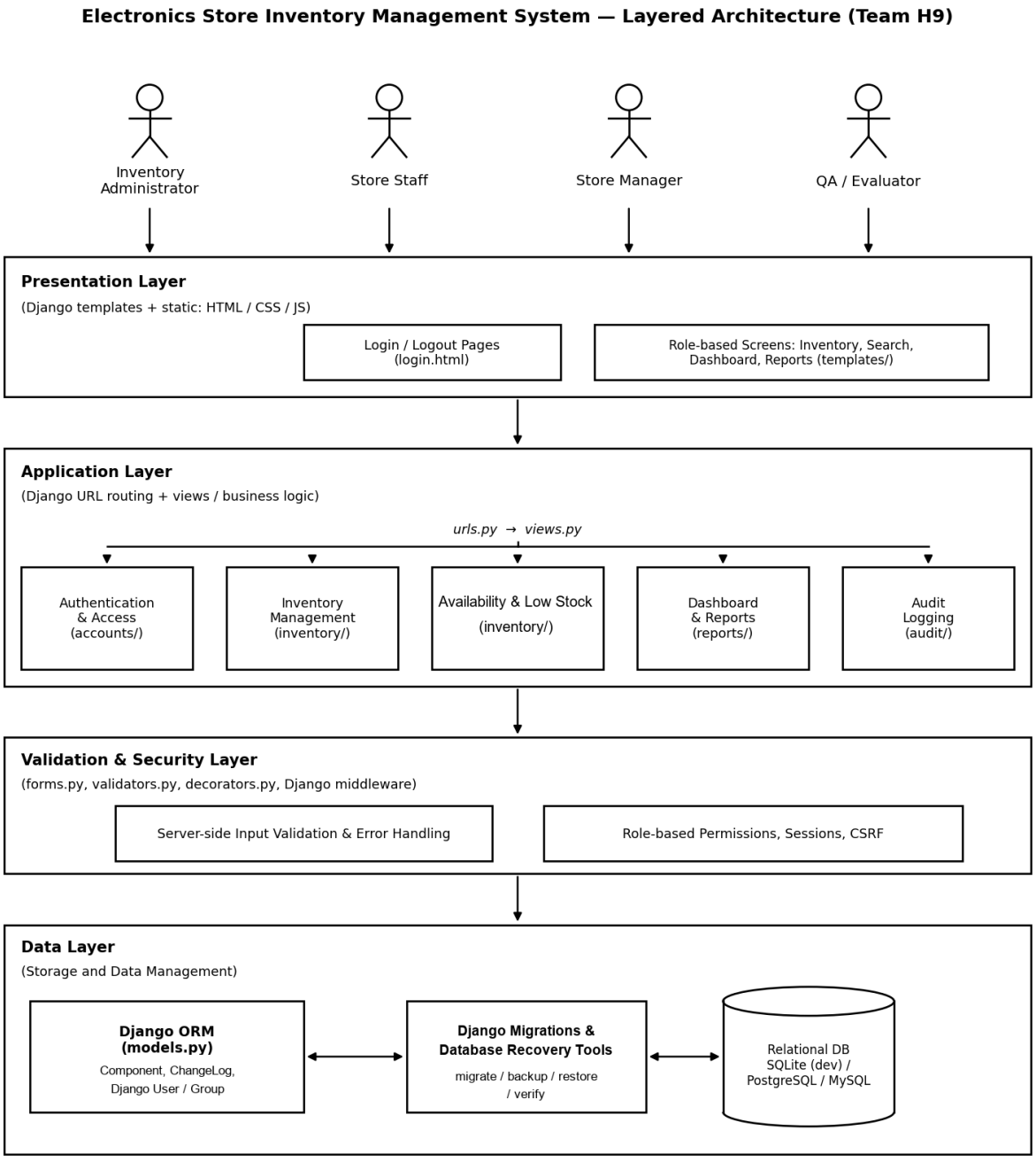


Figure 6 — Layered application architecture and module map

The former stock/ box represents the inventory app's availability and low-stock module; the audit model is ChangeLog; backup/restore operations are provided by the recovery tooling. The existing detailed component diagram remains Figure 1 in Section 3.3.

Appendix B — Implemented URL Interface Design

The application exposes server-rendered Django URLs rather than a separate REST API. Paths below are mounted by src/esims/urls.py and the app URL configurations. State-changing POST requests require CSRF protection; protected routes require authentication and role authorization as described in the relevant view.

Path	Method	Request	Response / error behavior
/accounts/login/	GET, POST	username, password, CSRF token	Login form; success redirects to /inventory/; invalid credentials re-render form.
/accounts/logout/	POST	CSRF token, authenticated session	Session is ended; redirects to login.
/accounts/profile/	GET	Authenticated session	Profile page; access follows profile view permission.
/inventory/	GET	q, category, authenticated session	Inventory list/search/filter; empty state when no match.
/inventory/add/	GET, POST	Component fields; CSRF token	Form or successful write; validation errors remain field-specific.
/inventory/{pk}/edit/	GET, POST	Editable fields; CSRF token	Form or update; invalid write preserves existing row.
/inventory/{pk}/delete/	GET, POST	Confirmation; CSRF token	Confirmation/deactivation; invalid object returns 404.
/inventory/low-stock/	GET	Authenticated session	Active components at/below minimum stock.
/reports/dashboard/	GET	Authorized session	Dashboard metrics and low-stock indicators.
/reports/	GET	category, availability	Filtered report page.
/reports/export/	GET	Report filters	CSV download for permitted roles.
/audit/	GET	Optional component filter	Change history for permitted administrators.

Standards basis

The design-description organization follows the supplied SAD template and IEEE Std 1016-2009 (Software Design Descriptions), with architecture-description concepts referenced from ISO/IEC/IEEE 42010. The document is a course project design description and does not claim conformance certification.

The corrected architecture figure is based on the Phase_1/Architecture Diagram.png referenced by the course team. The original image remains preserved in docs/Phase_1_Reference/.

Appendix C — Role and Database Scope

This table records the current implementation baseline observed in src/accounts/roles.py, the protected views, and src/esims/settings.py. It is not a substitute for team approval of future policy changes.

Role	Inventory view	Add / edit / deactivate	Dashboard / reports	Audit history	Django auth user permissions
Inventory Administrator	Yes	Yes	Yes	Yes	Granted by role group
Store Staff	Yes	No	No	No	No
Store Manager	Yes	No	Yes	No	No

The Administrator group receives inventory write and Django auth user-model permissions. Server-side view checks restrict inventory writes and audit-history access to that role. Staff and Manager can view inventory; Manager is allowed on dashboard/report views. The repository has no custom user-management page, so group permissions must not be described as a separate application UI.

Current database scope: the application uses a file-backed SQLite database for local development and testing. src/esims/settings.py reads ESIMS_DB_PATH when set and otherwise uses BASE_DIR/db.sqlite3. No PostgreSQL or MySQL deployment configuration is present in this repository. Production database selection is deferred because production deployment is outside the current course demo scope.